

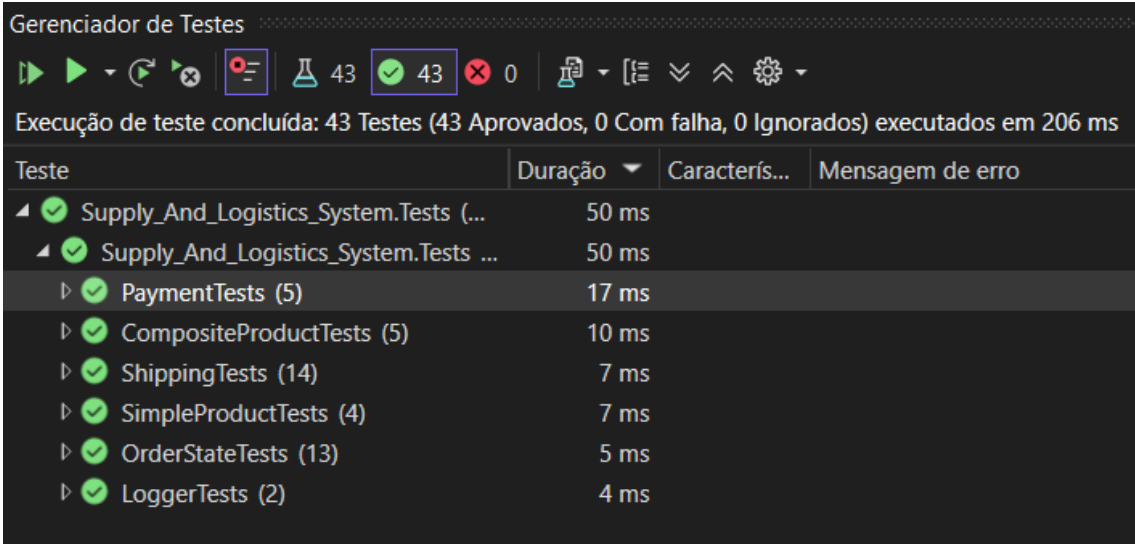
Adı Soyadı: LUCAS ISAAC CASSOMA KATALAHALI – 1A Grubu

TEST RAPORU

Kritik metotlar için yazılmış Birim Testleri (Unit Tests) ve çıktıları.

Test Özeti

- **Toplam Test Sayısı:** 43
- **Başarılı:** 43
- **Başarısız:** 0
- **Toplam Süre:** 206 ms



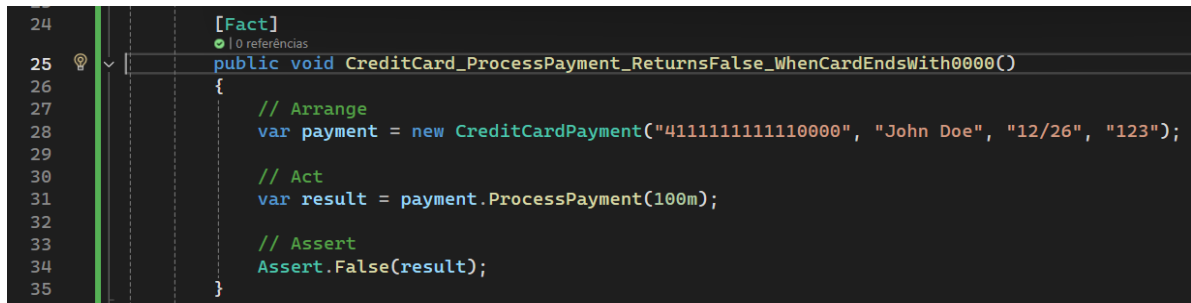
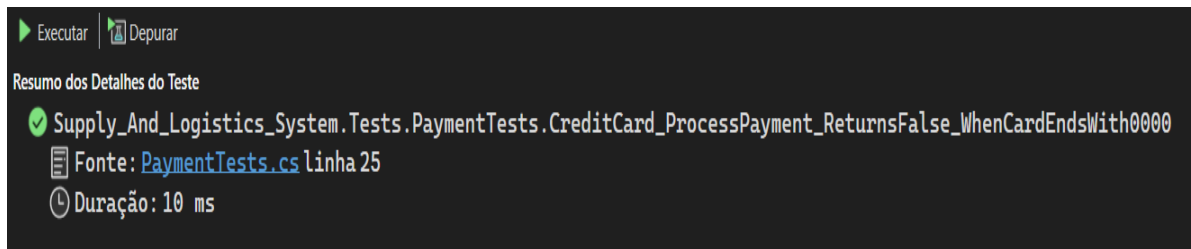
Gerenciador de Testes

Execução de teste concluída: 43 Testes (43 Aprovados, 0 Com falha, 0 Ignorados) executados em 206 ms

Teste	Duração	Caracterís...	Mensagem de erro
✓ Supply_And_Logistics_System.Tests (...)	50 ms		
✓ Supply_And_Logistics_System.Tests ...	50 ms		
▶ ✓ PaymentTests (5)	17 ms		
▶ ✓ CompositeProductTests (5)	10 ms		
▶ ✓ ShippingTests (14)	7 ms		
▶ ✓ SimpleProductTests (4)	7 ms		
▶ ✓ OrderStateTests (13)	5 ms		
▶ ✓ LoggerTests (2)	4 ms		

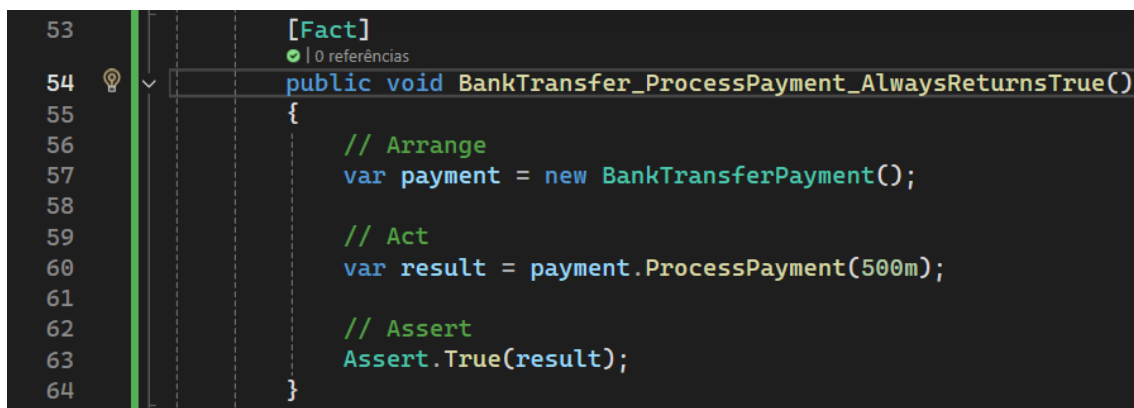
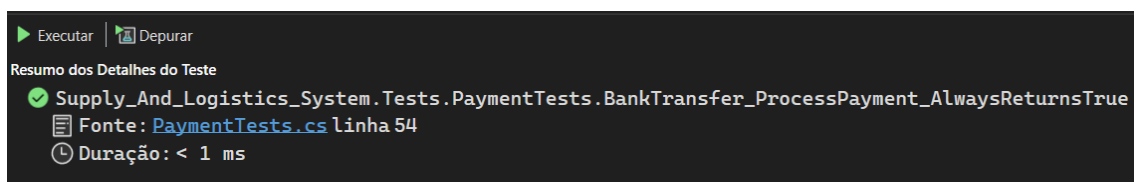
Aşağıda, bu testlerin bazılarının açıklamaları ve çıktıları yer almaktadır.

TEST ANALIZI: KREDİ KARTI HATA SİMÜLASYONU



- Dosya: PaymentTests.cs
- Satır: 25
- Açıklama: Bu test, kredi kartı ödeme stratejisinin (CreditCardPayment) başarısızlık senaryosunu doğrular. Sistemde, kart numarası "0000" ile biten durumlar bir hata simülasyonu olarak kurgulanmıştır.

TEST ANALIZI: BANKA HAVALE Sİ ÖDEME DOĞRULAMASI



- Dosya: PaymentTests.cs

- Satır: 54
- Açıklama: Bu test, BankTransferPayment stratejisinin ödeme sürecini doğrular. Banka havalesi yöntemiyle yapılan işlemlerin her zaman başarılı (true) döndüğü simüle edilmiştir.

TEST ANALIZI: GEÇERLİ KREDİ KARTI ÖDEME DOĞRULAMASI

The screenshot shows the Visual Studio Test Explorer on the left, indicating a successful test run for 'Supply_And_Logistics_System.Tests.PaymentTests.CreditCard_ProcessPayment_ReturnsTrue_WhenValidCard' at line 12 of 'PaymentTests.cs' with a duration of less than 1 ms. The right pane shows the corresponding test code in the Code Editor:

```

11 [Fact]
12 public void CreditCard_ProcessPayment_ReturnsTrue_WhenValidCard()
13 {
14     // Arrange
15     var payment = new CreditCardPayment("411111111111234", "John Doe", "12/26", "123");
16
17     // Act
18     var result = payment.ProcessPayment(100m);
19
20     // Assert
21     Assert.True(result);
22 }

```

- Dosya: PaymentTests.cs
- Satır: 12
- Açıklama: Bu test, CreditCardPayment stratejisinin başarılı ödeme senaryosunu doğrular. Geçerli formatta bir kart numarası ve bilgilerle yapılan işlemin true döndürdüğünü teyit eder.

TEST ANALIZI: BİLEŞİK ÜRÜN FİYAT HESAPLAMA DOĞRULAMASI

The screenshot shows the Visual Studio Test Explorer with a successful test run for 'Supply_And_Logistics_System.Tests.CompositeProductTests.GetPrice_ReturnsSumOfComponentPrices' at line 9 of 'CompositeProductTests.cs' with a duration of 7 ms.

```

8      [Fact]
9      public void GetPrice_ReturnsSumOfComponentPrices()
10     {
11         // Arrange
12         var ram = new SimpleProduct { Name = "RAM", Price = 50m, Stock = 10, Threshold = 2 };
13         var cpu = new SimpleProduct { Name = "CPU", Price = 100m, Stock = 10, Threshold = 2 };
14         var pc = new CompositeProduct { Name = "PC" };
15         pc.AddComponent(ram);
16         pc.AddComponent(cpu);
17
18         // Act
19         var result = pc.GetPrice();
20
21         // Assert
22         Assert.Equal(150m, result);
23     }

```

- Dosya: CompositeProductTests.cs
- Satır: 9
- Açıklama: Bu test, CompositeProduct nesnesinin fiyat hesaplama mantığını doğrular. Bileşik bir ürünün fiyatının, onu oluşturan tüm alt bileşenlerin (RAM ve CPU) fiyatlarının toplamına eşit olduğunu teyit eder.

TEST ANALIZİ: BİLEŞEN EKLEME VE KOLEKSİYON YÖNETİMİ DOĞRULAMASI

▶ Executar | Depurar

Resumo dos Detalhes do Teste

✓ Supply_And_Logistics_System.Tests.CompositeProductTests.AddComponent_IncreasesComponentCount

Fonte: [CompositeProductTests.cs](#) linha 74

🕒 Duração: 2 ms

```

73     [Fact]
74     public void AddComponent_IncreasesComponentCount()
75     {
76         // Arrange
77         var pc = new CompositeProduct { Name = "PC" };
78         var ram = new SimpleProduct { Name = "RAM", Price = 50m, Stock = 10, Threshold = 2 };
79
80         // Act
81         pc.AddComponent(ram);
82
83         // Assert
84         Assert.Single(pc.Components);
85     }

```

- Dosya: CompositeProductTests.cs
- Satır: 74
- Açıklama: Bu test, CompositeProduct sınıfının içine yeni bir parça ekleme işlevini doğrular. Bir bileşik ürüne (PC) yeni bir bileşen (RAM) eklendiğinde, listenin doğru şekilde güncellendiğini ve bileşen sayısının arttığını teyit eder.

TEST ANALIZİ: ARAS KARGO ADAPTÖRÜ MALİYET HESAPLAMA DOĞRULAMASI

```
▶ Executar | Depurar
Resumo dos Detalhes do Teste
✓ Supply_And_Logistics_System.Tests.ShippingTests.ArasAdapter_CalculateCost_ReturnsPositiveValue
  Fonte: ShippingTests.cs linha 14
  Duração: 4 ms
```

```
13 [Fact]
14 public void ArasAdapter_CalculateCost_ReturnsPositiveValue()
15 {
16     var carrier = new ArasAdapter();
17     var cost = carrier.CalculateCost(5, 100);
18     Assert.True(cost > 0);
19 }
```

- Dosya: ShippingTests.cs
- Satır: 14
- Açıklama: Bu test, ArasAdapter sınıfının kargo maliyetini doğru bir şekilde hesaplayıp hesaplamadığını kontrol eder. Belirli bir ağırlık ve mesafe için hesaplanan tutarın sıfırdan büyük (pozitif) bir değer olduğunu teyit eder.

TEST ANALİZİ: ÇOKLU DEKORATÖR KULLANIMI VE MALİYET HESAPLAMA DOĞRULAMASI

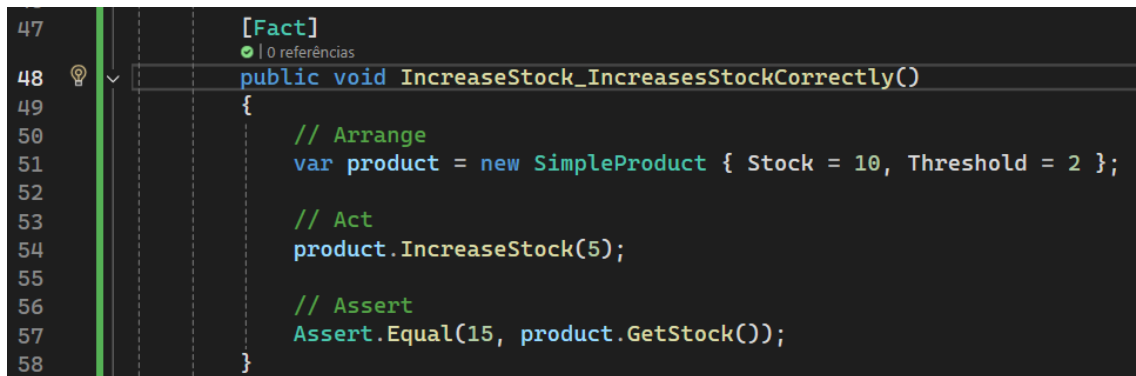
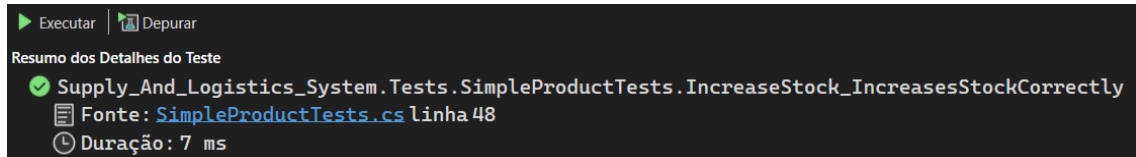
```
▶ Executar | Depurar
Resumo dos Detalhes do Teste
✓ Supply_And_Logistics_System.Tests.ShippingTests.BothDecorators_AddSeventyFiveEuros
  Fonte: ShippingTests.cs linha 107
  Duração: < 1 ms
```

```
106 [Fact]
107 public void BothDecorators_AddSeventyFiveEuros()
108 {
109     var baseCarrier = new ArasAdapter();
110     var baseCost = baseCarrier.CalculateCost(5, 100);
111
112     // aplica os dois decorators
113     ICarrier decorated = new InsuranceDecorator(baseCarrier);
114     decorated = new FragileProtectionDecorator(decorated);
115
116     var finalCost = decorated.CalculateCost(5, 100);
117
118     Assert.Equal(baseCost + 75m, finalCost); // 50 + 25
119 }
```

- Dosya: ShippingTests.cs
- Satır: 107
- Açıklama: Bu test, kargo işlemlerinde birden fazla ek hizmetin (Sigorta ve Hassas Ürün Koruması) maliyete doğru şekilde yansıtılıp yansıtılmadığını

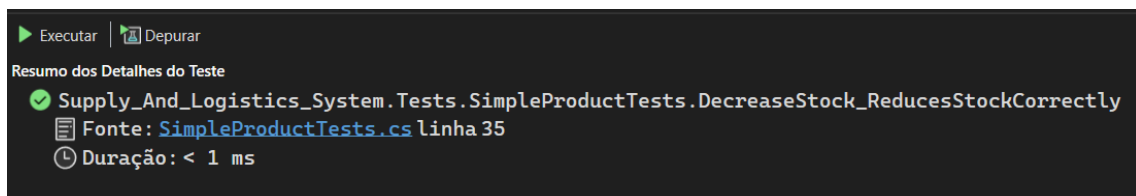
kontrol eder. Temel kargo ücretine toplamda 75 birim (50 sigorta + 25 koruma) eklendiğini teyit eder.

TEST ANALIZI: BASIT ÜRÜN STOK ARTIRIMI DOĞRULAMASI



- Dosya: SimpleProductTests.cs
- Satır: 48
- Açıklama: Bu test, SimpleProduct sınıfındaki stok artırma (IncreaseStock) metodunun temel işlevselliğini doğrular. Mevcut stoğu 10 olan bir ürüne 5 birim eklendiğinde, yeni stok değerinin doğru bir şekilde 15 olduğunu teyit eder.

TEST ANALIZI: BASIT ÜRÜN STOK AZALTMA DOĞRULAMASI



```

33
34 [Fact]
35 public void DecreaseStock_ReducesStockCorrectly()
36 {
37     // Arrange
38     var product = new SimpleProduct { Stock = 10, Threshold = 2 };
39
40     // Act
41     product.DecreaseStock(3);
42
43     // Assert
44     Assert.Equal(7, product.GetStock());
45 }
46

```

- Dosya: SimpleProductTests.cs
- Satır: 35
- Açıklama: Bu test, SimpleProduct sınıfındaki stok azaltma (DecreaseStock) metodunun işlevselliğini doğrular. Başlangıçta 10 birim stoğu olan bir üründen 3 birim düşüldüğünde, kalan stok miktarının 7 olduğunu teyit eder.

TEST ANALIZI: TRANSIT DURUMUNDA İADE TALEBİ DOĞRULAMASI

▶ Executar | Depurar

Resumo dos Detalhes do Teste

✓ Supply_And_Logistics_System.Tests.OrderStateTests.InTransitState_RequestReturn_MovesToReturnState

Fonte: [OrderStateTests.cs](#) linha 108

Duração: 4 ms

```

106
107 [Fact]
108 public void InTransitState_RequestReturn_MovesToReturnState()
109 {
110     var order = CreateOrder(new InTransitState());
111     order.LoadState();
112     order.RequestReturn();
113     Assert.Equal("ReturnState", order.StateName);
114 }
115

```

- Dosya: OrderStateTests.cs
- Satır: 108
- Açıklama: Bu test, InTransitState bir sipariş için iade talebi oluşturulduğunda, siparişin doğru bir şekilde iade durumuna ReturnState geçip geçmediğini kontrol eder.

TEST ANALIZI: TESLİM EDİLMİŞ SİPARİŞ İPTAL ENGELLEME DOĞRULAMASI

The screenshot shows the Visual Studio Test Explorer at the top, indicating a successful test run for 'Supply_And_Logistics_System.Tests.OrderStateTests.DeliveredState_Cancel_ThrowsException' at line 120 of 'OrderStateTests.cs' with a duration of 1 ms. Below, the code editor shows the test method:

```
[Fact]
public void DeliveredState_Cancel_ThrowsException()
{
    var order = CreateOrder(new DeliveredState());
    order.LoadState();
    Assert.Throws<InvalidOperationException>(() => order.Cancel());
}
```

- Dosya: OrderStateTests.cs
- Satır: 120
- Açıklama: Bu test, teslim edilmiş (DeliveredState) bir siparişin iptal edilmeye çalışılması durumunda sistemin beklenen hata tepkisini verip vermediğini ölçer. Teslimat tamamlandığı için iptal işleminin artık mümkün olmadığını ve bir InvalidOperationException fırlatıldığını doğrular.

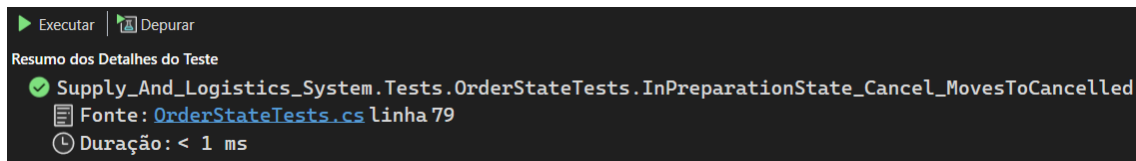
TEST ANALIZI: TESLİM EDİLMİŞ SIPARIŞTE İADE TALEBİ KISITLAMASI

The screenshot shows the Visual Studio Test Explorer at the top, indicating a successful test run for 'Supply_And_Logistics_System.Tests.OrderStateTests.DeliveredState_RequestReturn_ThrowsException' at line 128 of 'OrderStateTests.cs' with a duration of less than 1 ms. Below, the code editor shows the test method:

```
[Fact]
public void DeliveredState_RequestReturn_ThrowsException()
{
    var order = CreateOrder(new DeliveredState());
    order.LoadState();
    Assert.Throws<InvalidOperationException>(() => order.RequestReturn());
}
```

- Dosya: OrderStateTests.cs
- Satır: 128
- Açıklama: Bu test, sipariş teslim edildikten sonra (DeliveredState) doğrudan sistem üzerinden iade talebi yapılamayacağını doğrular. Bu durumda sistemin bir InvalidOperationException fırlatarak işlemi engellediğini teyit eder (Bu senaryoda iade için müşteri hizmetleriyle iletişime geçilmesi kuralı simüle edilmiştir).

TEST ANALIZI: HAZIRLIK AŞAMASINDAKİ SİPARİŞİN İPTALI DOĞRULAMASI



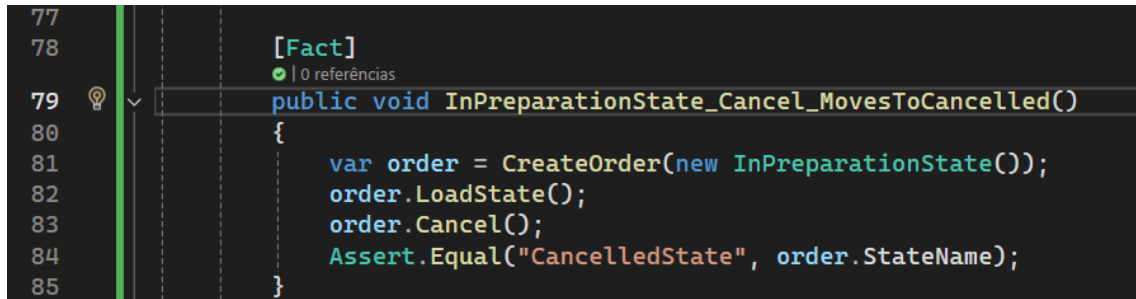
Executar | Depurar

Resumo dos Detalhes do Teste

✓ Supply_And_Logistics_System.Tests.OrderStateTests.InPreparationState_Cancel_MovesToCancelled

Fonte: [OrderStateTests.cs](#) linha 79

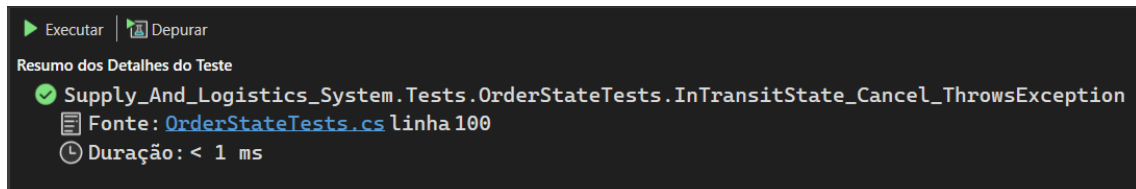
Duração: < 1 ms



```
77
78 [Fact]
79 public void InPreparationState_Cancel_MovesToCancelled()
80 {
81     var order = CreateOrder(new InPreparationState());
82     order.LoadState();
83     order.Cancel();
84     Assert.Equal("CancelledState", order.StateName);
85 }
```

- Dosya: OrderStateTests.cs
- Satır: 79
- Açıklama: Bu test, "Hazırlanıyor" (InPreparationState) aşamasındaki bir siparişin iptal edilme sürecini doğrular. Sipariş henüz kargoya verilmediği için bu aşamada iptal işleminin başarılı olması ve durumun "İptal Edildi" (CancelledState) olarak güncellenmesi gerektiğini teyit eder.

TEST ANALIZI: TRANSIT AŞAMASINDAKİ SİPARİŞ İPTAL KISITLAMASI



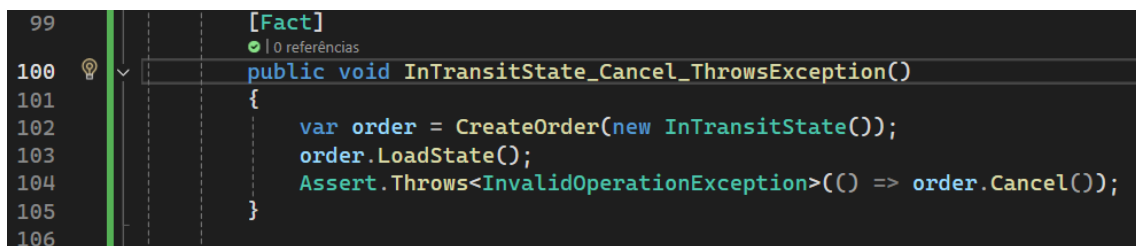
Executar | Depurar

Resumo dos Detalhes do Teste

✓ Supply_And_Logistics_System.Tests.OrderStateTests.InTransitState_Cancel_ThrowsException

Fonte: [OrderStateTests.cs](#) linha 100

Duração: < 1 ms



```
99 [Fact]
100 public void InTransitState_Cancel_ThrowsException()
101 {
102     var order = CreateOrder(new InTransitState());
103     order.LoadState();
104     Assert.Throws<InvalidOperationException>(() => order.Cancel());
105 }
106
```

- Dosya: OrderStateTests.cs
- Satır: 100
- Açıklama: Bu test, kargoya verilmiş ve InTransitState bir siparişin iptal edilmeye çalışılması durumunda sistemin koruma mekanizmasını doğrular. Sipariş fiziksel olarak dağıtımda olduğu için iptal işleminin yapılamayacağını

ve sistemin `InvalidOperationException` fırlatarak bu işlemi engellediğini teyit eder.